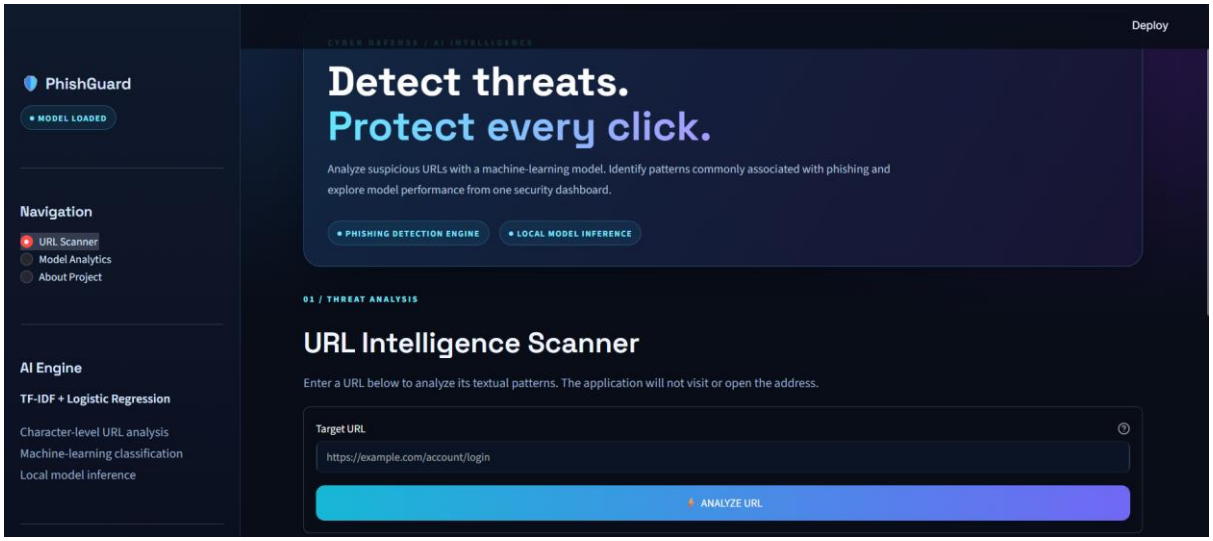
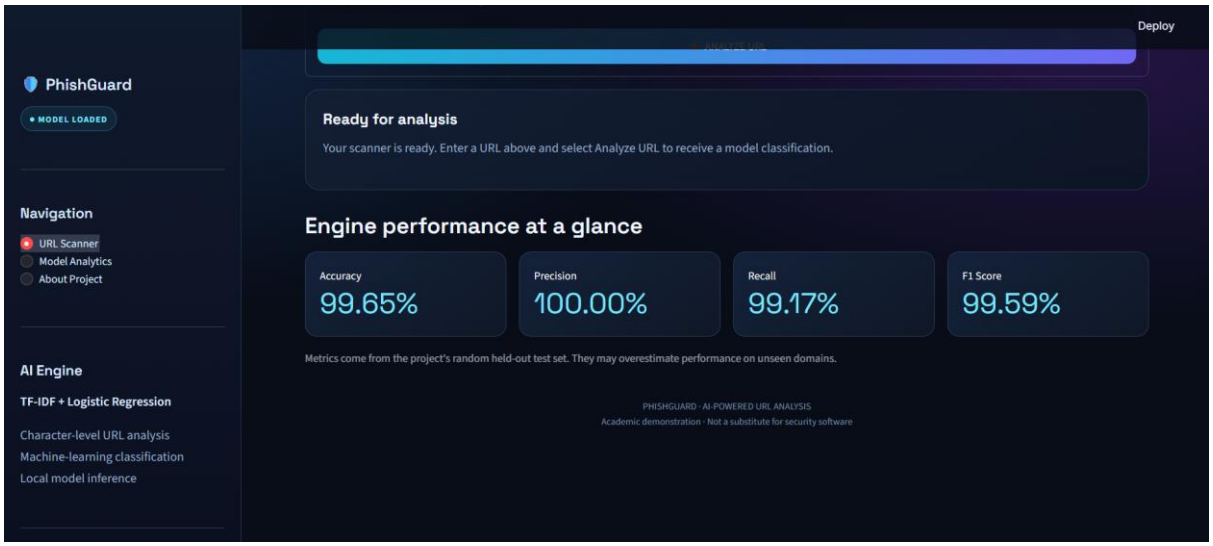
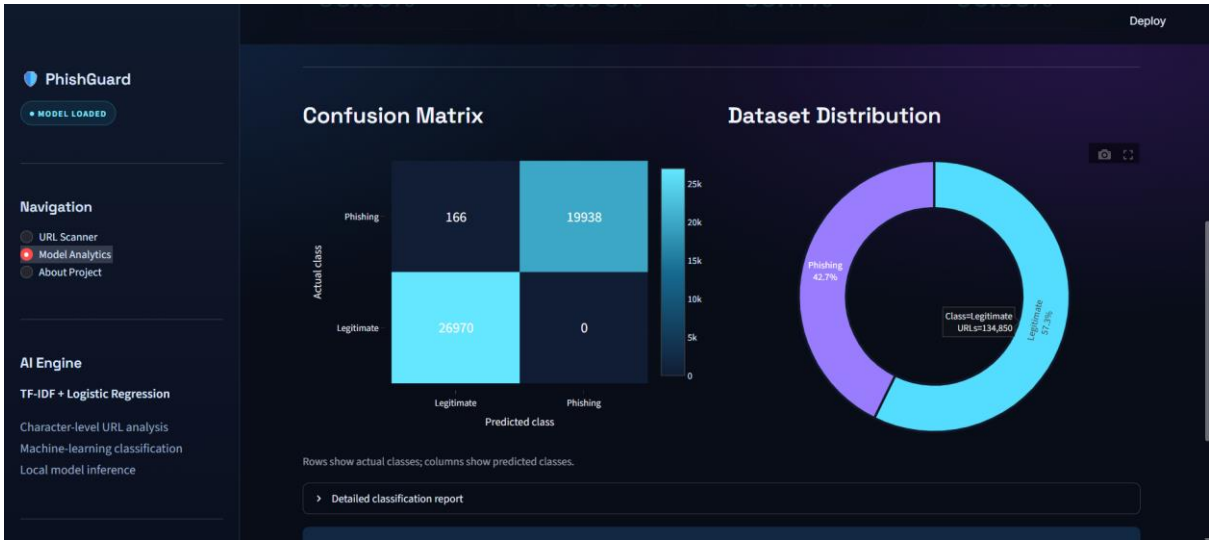
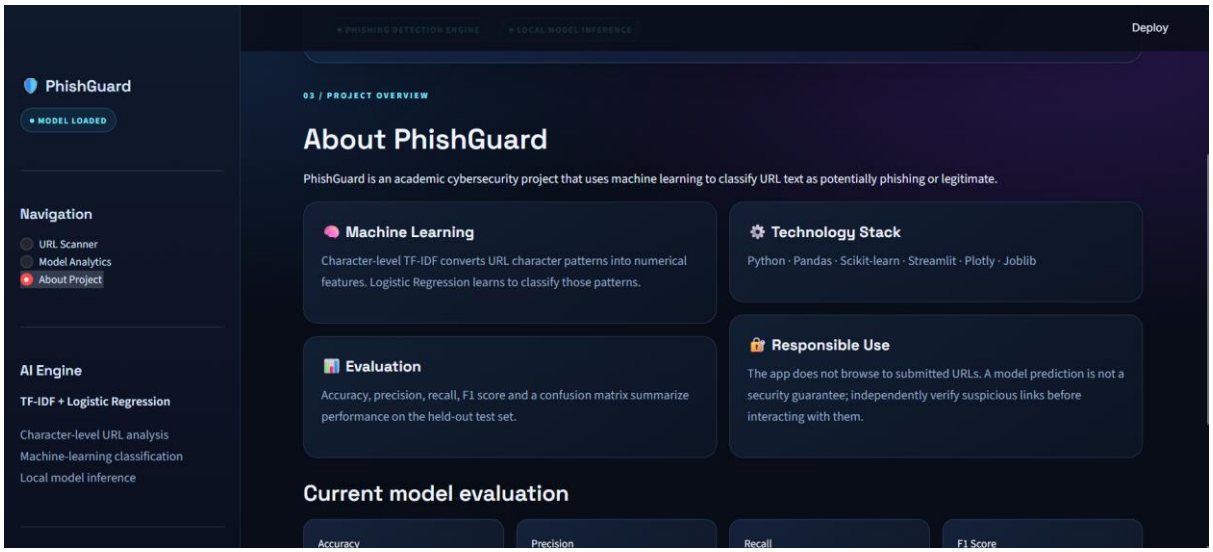

Deploy
PhishGuard
• MODEL LOADED
Navigation
URL Scanner
Model Analytics
About Project
AI Engine
TF-IDF + Logistic Regression
Character-level URL analysis
Machine-learning classification
Local model inference
Detect threats.
Protect every click.
Analyze suspicious URLs with a machine-learning model. Identify patterns commonly associated with phishing and explore model performance from one security dashboard.
• PHISHING DETECTION ENGINE
• LOCAL MODEL INFERENCE
01 / THREAT ANALYSIS
URL Intelligence Scanner
Enter a URL below to analyze its textual patterns. The application will not visit or open the address.
Target URL
https://example.com/account/login
ANALYZE URL


Deploy
PhishGuard
• MODEL LOADED
Navigation
URL Scanner
Model Analytics
About Project
AI Engine
TF-IDF + Logistic Regression
Character-level URL analysis
Machine-learning classification
Local model inference
Ready for analysis
Your scanner is ready. Enter a URL above and select Analyze URL to receive a model classification.
Engine performance at a glance
Accuracy
99.65%
Precision
100.00%
Recall
99.17%
F1 Score
99.59%
Metrics come from the project's random held-out test set. They may overestimate performance on unseen domains.
PHISHGUARD · AI-POWERED URL ANALYSIS
Academic demonstration · Not a substitute for security software





App.py

```
import json
from pathlib import Path

import joblib
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

# ----------------------------------------------
```

```
# PAGE CONFIGURATION
# ------------------------------------------------
st.set_page_config(
    page_title="PhishGuard | Cybersecurity AI",
    page_icon="🛡️",
    layout="wide",
    initial_sidebar_state="expanded",
)

ARTIFACTS = Path("artifacts")
MODEL_PATH = ARTIFACTS / "phishing_model.joblib"
METRICS_PATH = ARTIFACTS / "metrics.json"
COUNTS_PATH = ARTIFACTS / "class_counts.json"

# ------------------------------------------------
# CUSTOM UI STYLING
# ------------------------------------------------
st.markdown(
    """
    <style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&fami
ly=Space+Grotesk:wght@400;500;600;700&display=swap');

    .stApp {
        background:
            radial-gradient(ellipse at 10% 0%, #132644 0%, transparent 40%),
            radial-gradient(ellipse at 90% 10%, #241444 0%, transparent 38%),
            #080d18;
        color: #e8efff;
```

```
    font-family: 'Inter', sans-serif;
}

[data-testid="stHeader"] {
    background: rgba(8, 13, 24, 0.85);
}

[data-testid="stSidebar"] {
    background: linear-gradient(180deg, #101a2d, #0b1020);
    border-right: 1px solid #263650;
}

[data-testid="stSidebar"] * {
    color: #dce8ff;
}

h1, h2, h3 {
    font-family: 'Space Grotesk', sans-serif !important;
    color: #f3f7ff !important;
}

.hero {
    padding: 30px 30px 26px 30px;
    border: 1px solid rgba(87, 205, 255, 0.28);
    border-radius: 22px;
    background: linear-gradient(
        120deg,
        rgba(17, 39, 67, 0.96),
        rgba(24, 20, 52, 0.95)
```

```
    );
    margin-bottom: 24px;
    box-shadow: 0 12px 50px rgba(0, 0, 0, 0.22);
}

.eyebrow {
    color: #65e6ff;
    text-transform: uppercase;
    letter-spacing: 3px;
    font-size: 11px;
    font-weight: 800;
    margin-bottom: 12px;
}

.hero-title {
    font-family: 'Space Grotesk', sans-serif;
    font-size: clamp(34px, 5vw, 52px);
    font-weight: 700;
    line-height: 1.1;
    color: #ffffff;
    margin-bottom: 12px;
}

.gradient-text {
    background: linear-gradient(90deg, #64e8ff, #a99bff);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
```

```
.hero-desc {
    color: #b1c2dc;
    font-size: 15px;
    line-height: 1.8;
    max-width: 750px;
}

.section-label {
    color: #77e9ff;
    font-size: 11px;
    font-weight: 800;
    letter-spacing: 2px;
    text-transform: uppercase;
    margin: 8px 0 10px 0;
}

.panel {
    background: linear-gradient(
        145deg,
        rgba(19, 31, 52, 0.95),
        rgba(13, 21, 37, 0.95)
    );
    border: 1px solid #283a56;
    border-radius: 18px;
    padding: 22px;
    margin-bottom: 16px;
}

.panel-title {
```

```
    color: #f1f6ff;
    font-family: 'Space Grotesk', sans-serif;
    font-size: 19px;
    font-weight: 700;
    margin-bottom: 8px;
}

.muted {
    color: #9aacc7;
    font-size: 13px;
    line-height: 1.7;
}

.status-pill {
    display: inline-block;
    border: 1px solid #2c5670;
    background: rgba(32, 165, 203, 0.10);
    color: #7ce8ff;
    border-radius: 999px;
    padding: 6px 11px;
    font-size: 11px;
    font-weight: 700;
    letter-spacing: 1px;
}

.result-box {
    padding: 23px;
    border-radius: 18px;
    margin: 18px 0;
```

```
    background: #101b2d;
}

.result-danger {
    border: 1px solid #ff5978;
    background: linear-gradient(130deg, #301526, #171625);
}

.result-safe {
    border: 1px solid #30d9b0;
    background: linear-gradient(130deg, #10352f, #111e2d);
}

.result-title {
    font-family: 'Space Grotesk', sans-serif;
    font-size: 24px;
    font-weight: 700;
    margin-bottom: 8px;
}

div[data-testid="stMetric"] {
    background: linear-gradient(145deg, #14223a, #101827);
    border: 1px solid #2b3e5d;
    border-radius: 15px;
    padding: 18px 18px 14px 18px;
}

div[data-testid="stMetricLabel"] {
    color: #a8bdd9;
```

```
}

div[data-testid="stMetricValue"] {
    color: #78e8ff;
    font-family: 'Space Grotesk', sans-serif;
}

.stTextInput input {
    background: #0b1424 !important;
    color: #edf6ff !important;
    border: 1px solid #345273 !important;
    border-radius: 12px !important;
    min-height: 48px;
}

.stTextInput input:focus {
    border-color: #63e4ff !important;
    box-shadow: 0 0 0 1px #63e4ff !important;
}

.stButton button, .stFormSubmitButton button {
    background: linear-gradient(90deg, #17b8d6, #7168f5);
    color: white;
    border: none;
    border-radius: 11px;
    font-weight: 800;
    min-height: 45px;
    transition: all 0.2s ease;
}
```

```
    .stButton button:hover, .stFormSubmitButton button:hover {
        border: 1px solid #a5f1ff;
        box-shadow: 0 0 22px rgba(61, 211, 255, 0.25);
        color: white;
    }

    [data-testid="stAlert"] {
        border-radius: 12px;
    }

    hr {
        border-color: #25354d;
    }

    #MainMenu, footer {
        visibility: hidden;
    }

    .footer-note {
        text-align: center;
        color: #788ba8;
        font-size: 12px;
        padding: 24px 0 12px 0;
    }
    </style>
    """,
    unsafe_allow_html=True,
)
```

```
# ------------------------------------------------
# LOAD MODEL AND DATA
# ------------------------------------------------
if not MODEL_PATH.exists() or not METRICS_PATH.exists():
    st.error("Model files not found. Run `python train.py` first.")
    st.stop()

@st.cache_resource
def load_model():
    return joblib.load(MODEL_PATH)

model = load_model()

with open(METRICS_PATH, "r", encoding="utf-8") as file:
    metrics = json.load(file)

counts = {}
if COUNTS_PATH.exists():
    with open(COUNTS_PATH, "r", encoding="utf-8") as file:
        counts = json.load(file)

# ------------------------------------------------
# SIDEBAR
# ------------------------------------------------
with st.sidebar:
    st.markdown("## 🛡️ PhishGuard")
    st.markdown(
        '<span class="status-pill">● MODEL LOADED</span>',
```

```
    unsafe_allow_html=True,
)

st.markdown("---")
st.markdown("### Navigation")

section = st.radio(
    "Choose a section",
    [
        "URL Scanner",
        "Model Analytics",
        "About Project",
    ],
    label_visibility="collapsed",
)

st.markdown("---")
st.markdown("### AI Engine")
st.markdown("**TF-IDF + Logistic Regression**")
st.markdown(
    '<p class="muted">Character-level URL analysis<br>'
    'Machine-learning classification<br>'
    'Local model inference</p>',
    unsafe_allow_html=True,
)

st.markdown("---")
st.markdown("### Safety Notice")
st.markdown(
```

```
        '<p class="muted">This academic tool analyzes URL text. '
        'It does not open websites or guarantee that a URL is safe.</p>',
        unsafe_allow_html=True,
    )

# ---------------------------------------------
# HERO HEADER
# ---------------------------------------------
st.markdown(
    """
    <div class="hero">
        <div class="eyebrow">CYBER DEFENSE / AI INTELLIGENCE</div>
        <div class="hero-title">
            Detect threats.<br>
            <span class="gradient-text">Protect every click.</span>
        </div>
        <div class="hero-desc">
            Analyze suspicious URLs with a machine-learning model.
            Identify patterns commonly associated with phishing
            and explore model performance from one security dashboard.
        </div>
        <br>
        <span class="status-pill">● PHISHING DETECTION ENGINE</span>
          
        <span class="status-pill">● LOCAL MODEL INFERENCE</span>
    </div>
    """,
    unsafe_allow_html=True,
)
```

```
# ------------------------------------------------
# URL SCANNER
# ------------------------------------------------
if section == "URL Scanner":
    st.markdown(
        '<div class="section-label">01 / THREAT ANALYSIS</div>',
        unsafe_allow_html=True,
    )
    st.markdown("## URL Intelligence Scanner")
    st.markdown(
        '<p class="muted">Enter a URL below to analyze its textual patterns. '
        'The application will not visit or open the address.</p>',
        unsafe_allow_html=True,
    )

    with st.form("scanner_form"):
        url = st.text_input(
            "Target URL",
            placeholder="https://example.com/account/login",
            help="Enter the URL as text. Do not include private tokens.",
        )
        submitted = st.form_submit_button(
            "⚡ ANALYZE URL",
            type="primary",
            width="stretch",
        )

    if submitted:
```

```
cleaned_url = url.strip()

if not cleaned_url:
    st.warning("Enter a URL to begin the analysis.")
elif any(char.isspace() for char in cleaned_url):
    st.warning("Please remove spaces from the URL.")
else:
    prediction = int(model.predict([cleaned_url])[0])
    probabilities = model.predict_proba([cleaned_url])[0]
    classes = list(model.classes_)
    phishing_index = classes.index(1)
    phishing_score = float(probabilities[phishing_index])

    st.markdown("---")
    st.markdown("### Analysis report")

    if prediction == 1:
        st.markdown(
            f"""
            <div class="result-box result-danger">
                <div class="eyebrow">THREAT CLASSIFICATION</div>
                <div class="result-title">⚠ Potential Phishing URL</div>
                <div class="muted">
                    The model classified this URL as phishing.
                    Avoid entering credentials unless you independently
                    verify the destination.
                </div>
            </div>
            """,
```

```
        unsafe_allow_html=True,
    )
else:
    st.markdown(
        f"""
        <div class="result-box result-safe">
            <div class="eyebrow">MODEL CLASSIFICATION</div>
            <div class="result-title">□ Likely Legitimate</div>
            <div class="muted">
                The model classified this URL as legitimate.
                This does not prove that the website is safe.
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )

left, right = st.columns([1, 1])

with left:
    st.metric(
        "Estimated phishing score",
        f"{phishing_score * 100:.2f}%",
    )
    st.progress(phishing_score)
    st.caption(
        "Model-estimated probability, not a verified real-world risk."
    )
```

```
        with right:
            st.markdown(
                '<div class="panel">'
                '<div class="panel-title">Scan details</div>'
                f'<p class="muted"><b>URL length:</b> '
                f'{len(cleaned_url)} characters</p>'
                f'<p class="muted"><b>Prediction:</b> '
                f'{"Phishing" if prediction == 1 else "Legitimate"}</p>'
                '<p class="muted"><b>Analysis method:</b> '
                'Character TF-IDF + Logistic Regression</p>'
                '</div>',
                unsafe_allow_html=True,
            )

        with st.expander("View the analyzed URL"):
            st.code(cleaned_url, language=None)

else:
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">Ready for analysis</div>
            <p class="muted">
                Your scanner is ready. Enter a URL above and select
                Analyze URL to receive a model classification.
            </p>
        </div>
        """,
        unsafe_allow_html=True,
```

```
        )

    st.markdown("### Engine performance at a glance")
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Accuracy", f"{metrics['accuracy'] * 100:.2f}%")
    c2.metric("Precision", f"{metrics['precision'] * 100:.2f}%")
    c3.metric("Recall", f"{metrics['recall'] * 100:.2f}%")
    c4.metric("F1 Score", f"{metrics['f1_score'] * 100:.2f}%")

    st.caption(
        "Metrics come from the project's random held-out test set. "
        "They may overestimate performance on unseen domains."
    )

# ---------------------------------------------
# MODEL ANALYTICS
# ---------------------------------------------
elif section == "Model Analytics":
    st.markdown(
        '<div class="section-label">02 / MODEL TELEMETRY</div>',
        unsafe_allow_html=True,
    )
    st.markdown("## Model Performance & Analytics")
    st.markdown(
        '<p class="muted">Explore the training dataset distribution '
        'and the model evaluation results.</p>',
        unsafe_allow_html=True,
    )
```

```
c1, c2, c3, c4 = st.columns(4)
c1.metric("Accuracy", f"{metrics['accuracy'] * 100:.2f}%")
c2.metric("Precision", f"{metrics['precision'] * 100:.2f}%")
c3.metric("Recall", f"{metrics['recall'] * 100:.2f}%")
c4.metric("F1 Score", f"{metrics['f1_score'] * 100:.2f}%")

st.markdown("---")
left, right = st.columns(2)

with left:
    st.markdown("### Confusion Matrix")
    matrix = metrics.get("confusion_matrix", [])

    if matrix:
        labels = [
            "Legitimate",
            "Phishing",
        ]

        fig = go.Figure(
            data=go.Heatmap(
                z=matrix,
                x=labels,
                y=labels,
                colorscale=[
                    [0.0, "#111d35"],
                    [0.5, "#176b91"],
                    [1.0, "#65e8ff"],
                ],
```

```
                text=matrix,
                texttemplate="%{text}",
                textfont={"size": 18, "color": "#ffffff"},
                hovertemplate=(
                    "Actual: %{y}<br>"
                    "Predicted: %{x}<br>"
                    "Count: %{z}<extra></extra>"
                ),
            )
        )
        fig.update_layout(
            xaxis_title="Predicted class",
            yaxis_title="Actual class",
            paper_bgcolor="rgba(0,0,0,0)",
            plot_bgcolor="rgba(0,0,0,0)",
            font={"color": "#dce8ff"},
            margin=dict(l=10, r=10, t=25, b=10),
            height=380,
        )
        st.plotly_chart(fig, width="stretch")
        st.caption(
            "Rows show actual classes; columns show predicted classes."
        )

with right:
    st.markdown("### Dataset Distribution")

    if counts:
        distribution = pd.DataFrame(
```

```
    {
        "Class": list(counts.keys()),
        "URLs": list(counts.values()),
    }
)

fig = px.pie(
    distribution,
    names="Class",
    values="URLs",
    hole=0.65,
    color_discrete_sequence=["#55ddff", "#9a7cff"],
)
fig.update_traces(
    textposition="inside",
    textinfo="percent+label",
    marker=dict(
        line=dict(color="#101827", width=3)
    ),
)
fig.update_layout(
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    font={"color": "#dce8ff"},
    showlegend=False,
    margin=dict(l=10, r=10, t=25, b=10),
    height=380,
)
st.plotly_chart(fig, width="stretch")
```

```
        else:
            st.info("Dataset counts are not available.")

    with st.expander("Detailed classification report"):
        report = metrics.get("classification_report", {})
        report_df = pd.DataFrame(report).T
        st.dataframe(
            report_df.round(4),
            width="stretch",
        )

    st.info(
        "Evaluation caution: a random split can place related URLs in "
        "both training and test sets. Test on unseen domains before "
        "making claims about real-world detection performance."
    )

# ------------------------------------------------
# ABOUT PROJECT
# ------------------------------------------------
else:
    st.markdown(
        '<div class="section-label">03 / PROJECT OVERVIEW</div>',
        unsafe_allow_html=True,
    )
    st.markdown("## About PhishGuard")
    st.markdown(
        """
        PhishGuard is an academic cybersecurity project that uses machine
```

```
    learning to classify URL text as potentially phishing or legitimate.
    """
)

a, b = st.columns(2)

with a:
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">□ Machine Learning</div>
            <p class="muted">
                Character-level TF-IDF converts URL character patterns
                into numerical features. Logistic Regression learns
                to classify those patterns.
            </p>
        </div>
        """,
        unsafe_allow_html=True,
    )
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">📊 Evaluation</div>
            <p class="muted">
                Accuracy, precision, recall, F1 score and a confusion
                matrix summarize performance on the held-out test set.
            </p>
        </div>
```

```
        """,
        unsafe_allow_html=True,
    )

with b:
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">⚙ Technology Stack</div>
            <p class="muted">
                Python · Pandas · Scikit-learn · Streamlit · Plotly · Joblib
            </p>
        </div>
        """,
        unsafe_allow_html=True,
    )
    st.markdown(
        """
        <div class="panel">
            <div class="panel-title">🔒 Responsible Use</div>
            <p class="muted">
                The app does not browse to submitted URLs. A model
                prediction is not a security guarantee; independently
                verify suspicious links before interacting with them.
            </p>
        </div>
        """,
        unsafe_allow_html=True,
    )
```

```
    st.markdown("### Current model evaluation")
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Accuracy", f"{metrics['accuracy'] * 100:.2f}%")
    c2.metric("Precision", f"{metrics['precision'] * 100:.2f}%")
    c3.metric("Recall", f"{metrics['recall'] * 100:.2f}%")
    c4.metric("F1 Score", f"{metrics['f1_score'] * 100:.2f}%")

# ------------------------------------------------
# FOOTER
# ------------------------------------------------
st.markdown(
    """
    <div class="footer-note">
        PHISHGUARD · AI-POWERED URL ANALYSIS<br>
        Academic demonstration · Not a substitute for security software
    </div>
    """,
    unsafe_allow_html=True,
)
```

Train.py

```
import json
import os

import joblib
import pandas as pd
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
```

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
)

ARTIFACT_DIR = "artifacts"
os.makedirs(ARTIFACT_DIR, exist_ok=True)

print("Downloading the phishing URL dataset...")

# UCI PhiUSIIL Phishing URL Dataset
dataset = fetch_ucirepo(id=967)

X = dataset.data.features
y = dataset.data.targets

# Locate the URL and target columns.
url_columns = [
    col for col in X.columns
    if "url" in col.lower()
]
```

```
if not url_columns:
    raise ValueError(
        "Could not find a URL column. "
        f"Available columns: {list(X.columns)}"
    )

url_column = url_columns[0]

if y.shape[1] != 1:
    raise ValueError(
        f"Expected one target column, found: {list(y.columns)}"
    )

target_column = y.columns[0]

df = pd.DataFrame({
    "url": X[url_column].astype("string"),
    "label": pd.to_numeric(
        y[target_column], errors="coerce"
    ),
})

df = df.dropna(subset=["url", "label"])
df["url"] = df["url"].str.strip()
df = df[df["url"] != ""]
df = df.drop_duplicates(subset=["url"])

print("URL column:", url_column)
print("Target column:", target_column)
```

```
print("Dataset size:", len(df))
print("Target values:", sorted(df["label"].unique()))

# UCI dataset convention: 1 = legitimate, 0 = phishing.
# Convert it to 1 = phishing, 0 = legitimate.
df["label"] = df["label"].map({0: 1, 1: 0})
df = df.dropna(subset=["label"])
df["label"] = df["label"].astype(int)

if df["label"].nunique() != 2:
    raise ValueError(
        "Expected two classes. Check the dataset's target labels."
    )

X_train, X_test, y_train, y_test = train_test_split(
    df["url"],
    df["label"],
    test_size=0.2,
    random_state=42,
    stratify=df["label"],
)

print("Training model...")

model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            analyzer="char",
```

```
            ngram_range=(3, 5),
            max_features=100_000,
            dtype="float32",
        ),
    ),
    (
        "classifier",
        LogisticRegression(
            max_iter=500,
            class_weight="balanced",
        ),
    ),
])

model.fit(X_train, y_train)

predictions = model.predict(X_test)

metrics = {
    "accuracy": float(accuracy_score(y_test, predictions)),
    "precision": float(
        precision_score(y_test, predictions, zero_division=0)
    ),
    "recall": float(
        recall_score(y_test, predictions, zero_division=0)
    ),
    "f1_score": float(
        f1_score(y_test, predictions, zero_division=0)
    ),
```

```
    "confusion_matrix": confusion_matrix(
        y_test, predictions, labels=[0, 1]
    ).tolist(),
    "classification_report": classification_report(
        y_test,
        predictions,
        labels=[0, 1],
        target_names=["Legitimate", "Phishing"],
        output_dict=True,
        zero_division=0,
    ),
    "test_samples": int(len(y_test)),
    "label_meanings": {
        "0": "Legitimate",
        "1": "Phishing",
    },
}

joblib.dump(model, os.path.join(ARTIFACT_DIR, "phishing_model.joblib"))

with open(
    os.path.join(ARTIFACT_DIR, "metrics.json"),
    "w",
    encoding="utf-8",
) as file:
    json.dump(metrics, file, indent=2)

with open(
    os.path.join(ARTIFACT_DIR, "class_counts.json"),
```

```
    "w",
    encoding="utf-8",
) as file:
    json.dump(
        {
            "Legitimate": int((df["label"] == 0).sum()),
            "Phishing": int((df["label"] == 1).sum()),
        },
        file,
        indent=2,
    )

print("\nTraining complete!")
print(f"Accuracy:  {metrics['accuracy']:.4f}")
print(f"Precision: {metrics['precision']:.4f}")
print(f"Recall:    {metrics['recall']:.4f}")
print(f"F1 score:  {metrics['f1_score']:.4f}")
print("\nConfusion matrix (rows=true, columns=predicted):")
print(metrics["confusion_matrix"])
print("\nClassification report:")
print(classification_report(
    y_test,
    predictions,
    labels=[0, 1],
    target_names=["Legitimate", "Phishing"],
    zero_division=0,
))
print("\nSaved model and metrics in the artifacts folder.")
```